

## **Arduino code**

```
```.cpp
#include <Servo.h>

// ===== PIN DEFINITIONS =====

// DC Motor
#define IN1 8
#define IN2 9
#define ENA 10

// Stepper
#define STEP_PIN 3
#define DIR_PIN 4

// Servo
#define SERVO_PIN 6
Servo myServo;

// Encoder
#define CLK 2
#define DT 5
volatile int encoderPos = 0;

// Current Sensor
#define CURRENT_PIN A0

// ===== SYSTEM VARIABLES =====

// Current monitoring
float currentVoltage = 0;
float CURRENT_THRESHOLD = 2.5;

// Fault flags
bool dcFault = false;
bool stepperFault = false;
bool servoFault = false;

// Timing
unsigned long faultTime = 0;
unsigned long recoveryDelay = 4000;
```

```

// Stepper monitoring
int lastEncoderPos = 0;
unsigned long lastStepTime = 0;

// ===== SETUP =====
void setup() {
  Serial.begin(9600);

  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(ENA, OUTPUT);

  pinMode(STEP_PIN, OUTPUT);
  pinMode(DIR_PIN, OUTPUT);

  pinMode(CLK, INPUT);
  pinMode(DT, INPUT);
  attachInterrupt(digitalPinToInterrupt(CLK), readEncoder, CHANGE);

  myServo.attach(SERVO_PIN);

  Serial.println("ADVANCED SELF-HEALING SYSTEM STARTED");
}

// ===== LOOP =====
void loop() {
  readCurrent();
  checkDCMotorFault();
  checkStepperFault();
  checkServoFault();

  if (dcFault || stepperFault || servoFault) {
    recoveryMode();
    return;
  }

  normalOperation();
}

// ===== NORMAL OPERATION =====
void normalOperation() {

```

```

// DC MOTOR
digitalWrite(IN1, HIGH);
digitalWrite(IN2, LOW);
analogWrite(ENA, 160);

// STEPPER MOTOR
digitalWrite(DIR_PIN, HIGH);

for (int i = 0; i < 50; i++) {
    digitalWrite(STEP_PIN, HIGH);
    delayMicroseconds(800);
    digitalWrite(STEP_PIN, LOW);
    delayMicroseconds(800);
}

// SERVO
myServo.write(90);

Serial.print("OK | Current: ");
Serial.print(currentVoltage);
Serial.print(" | Encoder: ");
Serial.println(encoderPos);

delay(200);
}

// ===== CURRENT SENSOR =====
void readCurrent() {
    int sensorValue = analogRead(CURRENT_PIN);
    currentVoltage = sensorValue * (5.0 / 1023.0);
}

// ===== DC MOTOR FAULT =====
void checkDCMotorFault() {
    if (currentVoltage > CURRENT_THRESHOLD && !dcFault) {
        dcFault = true;
        faultTime = millis();

        Serial.println("⚠ DC MOTOR OVERCURRENT!");

        analogWrite(ENA, 0); // stop motor
    }
}

```

```

// ===== STEPPER FAULT =====
void checkStepperFault() {

    if (millis() - lastStepTime > 1000) {

        if (encoderPos == lastEncoderPos) {
            stepperFault = true;
            faultTime = millis();

            Serial.println("⚠ STEPPER STALL DETECTED!");
        }

        lastEncoderPos = encoderPos;
        lastStepTime = millis();
    }
}

// ===== SERVO FAULT =====
void checkServoFault() {

    int pos = 90; // expected

    if (pos < 0 || pos > 180) {
        servoFault = true;
        faultTime = millis();

        Serial.println("⚠ SERVO RANGE ERROR!");
    }
}

// ===== RECOVERY MODE =====
void recoveryMode() {

    Serial.println("🔧 SYSTEM IN SELF-HEAL MODE");

    // Stop everything
    analogWrite(ENA, 0);

    // Wait before recovery
    if (millis() - faultTime > recoveryDelay) {

        Serial.println("🔄 ATTEMPTING RECOVERY...");
    }
}

```

```

    // Reset faults
    dcFault = false;
    stepperFault = false;
    servoFault = false;

    // Gradual restart
    analogWrite(ENA, 120);

    // Reverse stepper to release jam
    digitalWrite(DIR_PIN, !digitalRead(DIR_PIN));

    for (int i = 0; i < 100; i++) {
        digitalWrite(STEP_PIN, HIGH);
        delayMicroseconds(1000);
        digitalWrite(STEP_PIN, LOW);
        delayMicroseconds(1000);
    }

    // Reset servo
    myServo.write(90);

    Serial.println("✅ SYSTEM RECOVERED");
}

// ===== ENCODER =====
void readEncoder() {
    int dtValue = digitalRead(DT);

    if (dtValue == HIGH) {
        encoderPos++;
    } else {
        encoderPos--;
    }
}
}
...

```

## **Python Code**

```
%%  
clc;  
clear;  
%% Simulation Time  
sim_time = 15;  
ref_speed = 100;  
%% -----  
% 1. DC MOTOR PARAMETERS  
%% -----  
J1 = 0.01;  
B1 = 0.1;  
K1 = 0.01;  
R1 = 1;  
L1 = 0.5;  
num_dc = [K1];  
den_dc = [J1*L1 (J1*R1 + B1*L1) (B1*R1 + K1^2)];  
%% -----  
% 2. SERVO MOTOR PARAMETERS  
%% -----  
wn = 12; % natural frequency  
zeta = 0.7; % damping ratio  
num_servo = [wn^2];  
den_servo = [1 2*zeta*wn wn^2];  
%% -----  
% 3. STEPPER MOTOR PARAMETERS  
%% -----  
tau = 0.3; % time constant  
num_stepper = [1];  
den_stepper = [tau 1];  
disp('All motor models loaded');
```